

# Finanzas App



**Realizado por:** *Roberto Gómez Suárez*

**Curso:** *2ºDAM — 2025/2026*

**Centro:** *IES Luis Carrillo de Sotomayor*

# Índice

|                                               |           |
|-----------------------------------------------|-----------|
| <b>Descripción General</b>                    | <b>3</b>  |
| ¿Por qué usar FinanzasApp y no otro Software? | 3         |
| <b>Tecnologías Utilizadas</b>                 | <b>4</b>  |
| Herramientas de Desarrollo                    | 4         |
| Infraestructura                               | 4         |
| Backend                                       | 4         |
| Frontend                                      | 4         |
| <b>Arquitectura de la Solución</b>            | <b>5</b>  |
| Proyecto Core                                 | 6         |
| Proyecto Infraestructura                      | 9         |
| Proyecto API                                  | 10        |
| Proyecto UI                                   | 12        |
| <b>Base de Datos</b>                          | <b>15</b> |
| Descripción de las Tablas                     | 15        |
| Relaciones entre las Entidades                | 15        |
| Diagrama E/R                                  | 16        |
| Modelo Relacional                             | 17        |
| Normalización                                 | 17        |
| <b>Manual de Usuario</b>                      | <b>20</b> |
| Diagrama de Casos de Uso                      | 31        |
| <b>Pruebas</b>                                | <b>32</b> |
| Pruebas de la Interfaz                        | 32        |
| Pruebas de la Base de Datos y la API          | 36        |
| <b>Autenticación y Seguridad</b>              | <b>41</b> |
| <b>Despliegue</b>                             | <b>42</b> |
| <b>Conclusión</b>                             | <b>43</b> |
| <b>Webgrafía</b>                              | <b>44</b> |

## Descripción General

FinanzasApp es una aplicación de gestión de finanzas personales que nace con la idea de ofrecer a cualquier persona una herramienta sencilla pero completa para tener un control real de su dinero. La mayoría de personas no lleva un seguimiento ordenado de en qué gasta o cuánto ingresa cada mes, lo que dificulta tomar buenas decisiones económicas o cumplir objetivos de ahorro. FinanzasApp pretende resolver eso de forma accesible, sin necesidad de conocimientos de contabilidad ni de economía.

La aplicación permite registrar cada ingreso y gasto, organizarlos por categorías personalizadas, visualizar la distribución del gasto mediante gráficas, establecer objetivos financieros con seguimiento automático en tiempo real, y colaborar con otras personas a través de grupos compartidos que muestran un resumen financiero conjunto y un ranking de gastos entre los miembros.

Técnicamente, la aplicación está formada por dos partes bien diferenciadas. Por un lado, una API REST desarrollada con ASP.NET Core 9 que actúa como backend, gestionando toda la lógica de negocio, la autenticación mediante tokens JWT y el acceso a la base de datos. Por otro, una interfaz de usuario multiplataforma desarrollada con .NET MAUI 9 que se ejecuta tanto en Android como en Windows desde una única base de código.

La decisión de separar el backend en una API independiente no es solo técnica, sino también práctica: permite que la aplicación funcione desde cualquier dispositivo con conexión a internet, ya que los datos están siempre sincronizados en la nube. La API se despliega de forma independiente en Railway mediante Docker, y la base de datos MySQL está alojada en Aiven, un servicio gestionado en la nube que garantiza disponibilidad, copias de seguridad automáticas y conexiones cifradas mediante SSL.

La arquitectura del proyecto está organizada en cuatro capas bien separadas (Core, Infraestructura, API y UI), siguiendo principios de diseño que hacen el sistema mantenible, escalable y fácil de extender en el futuro sin necesidad de reescribir partes ya funcionales.

## ¿Por qué usar FinanzasApp y no otro Software?

- La característica más genuinamente diferencial es el **sistema de grupos con ranking**. La mayoría de apps de finanzas personales son completamente individuales. Ninguna de las grandes apps gratuitas tiene un espacio compartido donde varias personas vean sus gastos consolidados y comparen quién gasta más cada mes con un ranking. Es una funcionalidad social que aporta algo real para parejas, compañeros de piso o familias.
- El **sistema de notificaciones mixto** (persistentes + dinámicas sin configurar nada) también es diferencial en concepto. El usuario no tiene que configurar alertas manualmente: el sistema evalúa solo y avisa cuando algo va mal. El aviso de vencimiento con tiempo exacto en horas y minutos, o el aviso de exceso de gasto respecto al mes anterior, son detalles que apps como Fintonic no tienen en su versión gratuita.
- El **despliegue multiplataforma real** (Android y Windows desde el mismo código) sin coste adicional para el usuario es otro punto. Muchas alternativas son solo móvil o solo web.

# **Tecnologías Utilizadas**

## **Herramientas de Desarrollo**

- **Visual Studio 2022:** El IDE

## **Infraestructura**

- **Aiven:** EL Servicio de MySQL en la nube para la base de datos
- **Railway:** Es la Plataforma de Despliegue del backend mediante el contenedor Docker
- **Docker:** Es para la contenedorización de la API.
- **GitHub:** Para el control de versiones y como repositorio del código fuente

## **Backend**

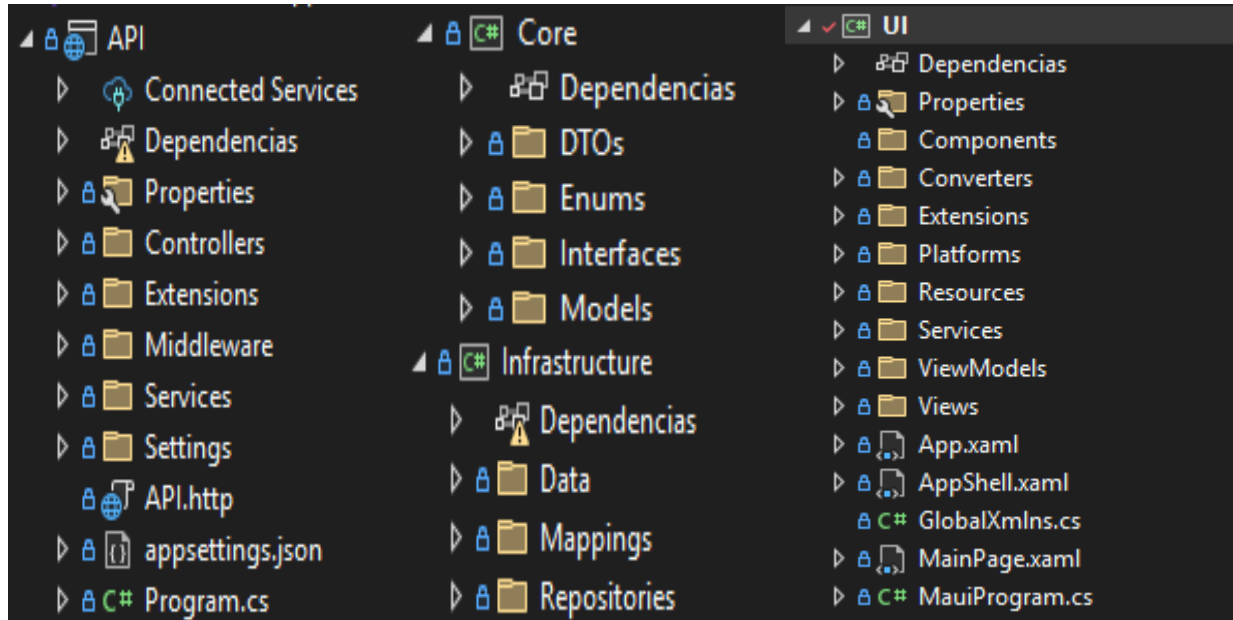
- **.Net 9:** Es el Framework para el desarrollo
- **ASP.NET Core Web API:** Para la construcción de los endpoints REST
- **Entity Framework Core 9:** ORM para el acceso a la base de datos
- **Pomelo.EntityFrameworkCore.MySQL 9.0.0:** Proveedor de EF Core para MySQL
- **MySQL 8:** Es el motor de la base de datos relacional
- **JWT (JSON Web Tokens):** Para la autenticación y la autorización
- **BCrypt:** Para el hash de las contraseñas
- **AutoMapper:** Para el mapeo entre las entidades y DTOs
- **Swagger:** Para las pruebas de la API

## **Frontend**

- **.NET MAUI 9:** Es el Framework multiplataforma para Android y Windows
- **CommunityToolkit.Mvvm:** Para la implementación de los MVVM
- **CommunityToolkit.Maui:** Los controles y las utilidades adicionales para MAUI
- **Microcharts.Maui:** Para las gráficas
- **SkiaSharp:** Es el motor de renderizado 2D usado internamente por Microcharts

# Arquitectura de la Solución

La solución del programa está organizada en 4 proyectos con una clara separación entre las capas.



Esta estructura consiste en que cada proyecto tiene una funcionalidad/propósito claro y las dependencias fluyen en una sola dirección. El proyecto Core no depende de nada. Infrastructure depende de Core. API depende de Core e Infrastructure. UI solo depende de Core para compartir los DTOs.

### Proyecto Core

El proyecto Core es la base de la solución. Contiene las definiciones de los modelos de dominio, las interfaces de los repositorios, los DTOs para la comunicación entre capas y las enumeraciones. Al no tener dependencias de otro proyecto, es el único proyecto al que se pueden referenciar todos los demás sin crear dependencias.

No tiene paquetes Nuget externos. Usa las librerías base de .NET 9.

### Carpeta DTOs

Los DTOs (Data Transfer Objects) son objetos planos que se usan para transportar los datos entre la API y el usuario, evitando exponer directamente los modelos de dominio y sus relaciones de navegación.

#### *DTOs de Autenticación*

- **LoginDto**: Contiene el Email y la contraseña para el inicio de sesión.
- **RegistroDto**: Contiene el Nombre, el email y la contraseña para el registro de usuarios
- **AuthResponseDto**: Contiene el Token JWT y los datos básicos para el usuario devueltos tras el login o registro.

#### *DTOs de Categoría*

- **CategoriaDto**: Contiene el ID, el nombre, el color y el ID del usuario
- **CrearCategoriaDto**: Contiene el Nombre y el color para crear una categoría

#### *DTOs de Grupo*

- **GrupoDto**: Contiene los datos del grupo incluyendo el número de miembros y si el usuario actual es el creador.
- **CrearGrupoDto**: Contiene el nombre para crear un grupo
- **MiembroGrupoDto**: Contiene los datos de un miembro dentro del ranking: nombre, ingresos y gastos del mes y la posición
- **ResumenGrupoDto**: Contiene los datos totales del grupo y la lista del ranking ordenado
- **SolicitudGrupoDto**: Contiene los datos de una solicitud para unirse a un grupo
- **InvitarUsuarioDto**: Contiene el Email del usuario al que se quiere invitar

#### *DTOs de Movimiento*

- **CrearMovimientoDto**: Contiene los campos necesarios para crear un movimiento
- **EditarMovimientoDto**: Contiene los campos necesarios para editar un movimiento
- **GraficaCategoriaDto**: Contiene el nombre de la categoría, total, porcentaje y el color para las gráficas circulares
- **GraficaMensualDto**: Contiene los datos de los meses para representarlos en la gráfica de barras
- **MovimientoDto**: Contiene todos los campos de un movimiento, incluyendo el nombre de la categoría

- **ResumenMesDto:** Contiene el total de ingresos y de gastos y el balance calculado para un mes concreto

### *DTOs de Notificación*

- **NotificacionDto:** Contiene todos los campos de una notificación incluyendo si es dinámica (las generadas en tiempo real) o persistente (guardadas en la base de datos).

### *DTOs de Objetivo*

- **CrearObjetivoDto:** Contiene los campos para crear o editar un objetivo
- **ObjetivoConProgresoDto:** Contiene un Objetivo con todos los campos calculados en tiempo real: la cantidad actual, el progreso, el estado, si está en peligro, si se ha cumplido o no y si está vencido o no.
- **ObjetivoDto:** Contiene la representación directa de un objetivo almacenado sin ningún cálculo.
- **TipoObjetivoDto:** Contiene el ID, el nombre y la descripción de un tipo de objetivo.

### *DTOs de Usuario*

- **CambiarPasswordDto:** Contiene los datos necesarios para cambiar la contraseña del usuario
- **UsuarioDto:** Contiene la representación directa de un usuario

## Carpeta Enums

- **PeriodoObjetivo:** Diario, Semanal, Mensual y Anual
- **TipoMovimiento:** Ingreso y Gasto
- **TipoObjetivoEnum:** Ahorro = 1, LimiteGasto = 2, MetaIngreso = 3 y ReduccionDeuda = 4

## Carpeta Interfaces

Aquí definimos los contratos que deben de cumplir los repositorios implementados en Infraestructure. Esto permite que la API dependa de abstracciones y no de las implementaciones concretas, facilitando el mantenimiento.

- **IRepository:** Es la interfaz genérica con unas operaciones básicas: ObtenerPorID, ObtenerTodos, Agregar, Actualizar y Eliminar.
- **ICategoriaRepository:** Añade el método ObtenerPorIdUsuario para obtener solo las categorías del usuario.
- **IGrupoRepository:** Son las operaciones sobre los grupos: ObtenerPorID, ObtenerGruposPorUsuarioId, Crear, Eliminar, EsMiembro, AgregarMiembro y ObtenerMiembros
- **IMovimientoRepository:** Añade los métodos: ObtenerPorIdUsuario, ObtenerPorUsuarioYMes, ObtenerPorTipo, ObtenerUltimos, ObtenerTotalPorTipoYMes, ObtenerPorUsuarioTipoYAnio, ObtenerPorUsuarioMesTipo y Buscar.
- **INotificacionRepository:** Define ObtenerPorUsuarioId, ExisteNotificacionObjetivo, Crear, MarcarLeida y MarcarTodasLeidas.

- **IObjetivoRepository:** Incluye ObtenerPorUsuarioId, ObtenerProximosAVencer y MarcarInactivo.
- **IRolRepository:** Añade ObtenerPorNombre y ObtenerPorId
- **ISolicitudGrupoRepository:** Gestiona las invitaciones a los grupos: ObtenerSolicitudesPendientesPorUsuario, ObtenerPorId, Crear, ActualizarEstado y ExisteSolicitudPendiente.
- **IUsuarioRepository:** Añade ObtenerPorId, ObtenerPorEmail, Añadir y Actualizar

## Carpeta Models

Contiene los modelos de dominio que se mapean directamente con las tablas de la base de datos:

- **Categoría:** Representa una categoría de movimientos creada por un usuario.
- **Grupo:** Representa un conjunto de usuarios que comparten información.
- **Movimiento:** Representa una transacción económica.
- **Notificacion:** Representa un aviso que se genera automáticamente por el sistema al evaluarse el estado de los objetivos
- **Objetivo:** Representa un objetivo financiero definido por el usuario
- **Rol:** Define los roles disponibles dentro de un grupo
- **SolicitudGrupo:** Representa las invitaciones para unirse a un grupo
- **TipoObjetivo:** Son los diferentes objetivos disponibles
- **Usuario:** Representa un usuario registrado en el sistema
- **UsuarioGrupo:** Es la tabla que hace de unión entre usuario y grupo

### Proyecto Infrastructure

En este proyecto, nos encargamos de todo lo relacionado con el acceso a los datos. Implementa las interfaces definidas en Core y configura el Entity Framework Core con el proveedor de MySQL

#### Paquetes NuGet

- **AutoMapper: 12.0.1:** Para el mapeo entre las entidades y los DTOs.
- **AutoMapper.Extensions.Microsoft.DependencyInjection 12.0.1:** Para la integración de AutoMapper con el sistema de inyección de las dependencias de .NET.
- **Microsoft.EntityFrameworkCore 9.0.0:** ORM principal.
- **Pomelo.EntityFrameworkCore.MySql 9.0.0:** Es el proveedor de EF Core para MySQL.

#### Carpeta Data

- **AppDbContext:** Es el contexto principal de Entity Framework Core. Aquí se definen los DbSet para todas las entidades y configura las relaciones, las restricciones y comportamientos mediante la API fuente de EF Core en el método OnModelCreating. Configura la conversión de los enums TipoMovimiento y PeriodoObjetivo a cadenas de texto para que se puedan almacenar en MySQL.

#### Carpeta Mappings

- **AutoMapperProfile:** Define todos los mapeos entre las entidades del dominio y sus DTOs correspondientes. Incluye mapeos personalizados para campos calculados como el caso de EstaVencido en los objetivos y CategoriaNombre en los movimientos

#### Carpeta Repositories

Cada repositorio implementa la interfaz correspondiente definida en Core. Todos reciben el AppDbContext por inyección de dependencias y utilizan \_contexto como el nombre de la variable para acceder a la base de datos. Los repositorios son: CategoriaRepository, GrupoRepository, MovimientoRepository, NotificacionRepository, ObjetivoRepository, RolRepository, SolicitudGrupoRepository y UsuarioRepository.

### Proyecto API

La API es el núcleo del backend. Expone los endpoints REST que consume la aplicación, gestiona la autenticación mediante JWT y coordina la lógica de negocio entre los repositorios.

#### Paquetes NuGet

- **AutoMapper 12.0.1:** Para el mapeo entre las entidades y los DTOs en los controladores.
- **AutoMapper.Extensions.Microsoft.DependencyInjection 12.0.1:** Para la integración con el contenedor de dependencias.
- **BCrypt.Net-Next 4.0.3:** Para el hash seguro de contraseñas.
- **Microsoft.AspNetCore.Authentication.JwtBearer 9.0.0:** Validación de los tokens JWT en cada petición.
- **Microsoft.EntityFrameworkCore.Design 9.0.0:** Las herramientas de diseño de EF Core para las migraciones.
- **Pomelo.EntityFrameworkCore.MySql 9.0.0:** Para proveer el MySQL para EF Core.
- **Swashbuckle.AspNetCore 6.9.0:** Para la generación automática de la documentación Swagger/OpenAPI.

#### Carpeta Controllers

Cada controlador se encarga de gestionar un recurso concreto de la API. Todos requieren autenticación JWT mediante el atributo Authorize, con la excepción del AuthController. El ID del usuario autenticado, se extrae del token en cada petición mediante la extensión ClaimsPrincipalExtensions.ObtenerUsuarioid(). Los CRUD que usamos son: AuthController, CategoríasController, GruposController, MovimientosController, NotificacionesController, ObjetivosController y UsuariosController.

#### Carpeta Extensions

- **ClaimsPrincipalExtensions:** Se encarga de obtener la información del usuario autenticado a partir de los claims presentes en el token JWT.

#### Carpeta Middleware

- **ErrorHandlingMiddleware:** Middleware global de la gestión de errores. Captura todas las excepciones no controladas y las convierte en respuestas JSON con el código HTTP correspondiente. Mapea NotFoundException a 404, UnauthorizedAccessException a 401, ArgumentException a 400 y cualquier otra a 500.

### Carpeta Services

Contiene la lógica de negocio más compleja, la cual no pertenece a un controlador en concreto.

- **AuthService / IAuthService:** Se encarga de gestionar el registro y el login. Verifica el email, hashlea la contraseña con BCrypt y genera el token JWT firmado con la clave secreta.
- **INotificacionService / NotificacionService:** Combina las notificaciones persistentes y las dinámicas. Genera notificaciones persistentes cuando un objetivo se cumple o se vence, usando un Semaphoreslim para evitar duplicados en peticiones simultáneas. Las notificaciones dinámicas (alertas de peligro, proximidad de vencimiento y exceso de gasto respecto al mes anterior) se calculan en cada petición y no se almacenan.
- **IObjetivoService / ObjetivoService:** Se encarga de evaluar el progreso de los objetivos en tiempo real. Carga todos los movimientos del usuario una sola vez y los filtra en memoria por el rango de fechas de cada objetivo para minimizar las llamadas a la base de datos. Implementa la lógica específica de evaluación para cada uno de los cuatro tipos de objetivo.

### Carpeta Settings

- **JwtSettings:** Esta clase representa la configuración necesaria para generar y validar tokens JWT en la aplicación, incluyendo la clave secreta, el issuer, el audience y el tiempo de expiración del token. Esta configuración se utiliza en el servicio de autenticación para crear tokens seguros y confiables que permitan a los usuarios autenticarse y acceder a los recursos protegidos de la API.

### Program

Es el punto de entrada de la aplicación. Se encarga de configurar todos los servicios del contenedor de dependencias, el middleware de la aplicación, la autenticación JWT, CORS, Swagger y la conexión a la base de datos. En producción el puerto se lee de la variable de entorno PORT para la compatibilidad con Railway.

### Proyecto UI

El proyecto UI contiene toda la parte de la interfaz de usuario de la aplicación, desarrollada con .NET MAUI siguiendo el patrón MVVM. Este patrón separa la vista (XAML), el estado y los comandos (ViewModel) y los datos remotos (a través del servicio ApiService), lo que hace el código mucho más mantenible y estable.

#### Paquetes NuGet

- **CommunityToolkit.Maui 12.0.0:** Controles adicionales y utilidades para MAUI.
- **CommunityToolkit.Mvvm 8.2.0:** Generación automática de código para MVVM.
- **Microcharts.Maui 1.0.1:** Librería de gráficas para MAUI basada en SkiaSharp. Proporciona gráficas de tipo donut y de barras.
- **Microsoft.Maui.Controls:** Controles base de MAUI incluidos automáticamente con el SDK.
- **Microsoft.Extensions.Logging.Debug 9.0.5:** Para el registro de logs en el modo debug.
- **SkiaSharp.Views.Maui.Controls 3.119.2:** Motor de renderizado 2D necesario para los Microcharts.

#### Carpeta Converts

Los converts nos permiten transformar valores en el binding XAML sin necesidad de lógica en el ViewModel:

- **TipoMovimientoToColorConverter:** Devuelve el color de fondo del indicador según si el movimiento es ingreso (verde) o gasto (rojo).
- **TipoMovimientoToTextColorConverter:** Devuelve el color del texto de la cantidad según el tipo.
- **TipoMovimientoToEmojiConverter:** Devuelve el emoji correspondiente al tipo de movimiento.
- **BoolToColorConverter:** Convierte un booleano en uno de dos colores configurables. Se usa para los botones de selección activa en gráficas y creación de movimientos.
- **BoolToEmojiConverter:** Devuelve un emoji de  o  según si el balance es positivo o negativo.
- **BoolToStringConverter:** Convierte un booleano en una de dos cadenas de texto configurables.
- **InvertedBoolConverter:** Invierte un valor booleano. Se usa para la visibilidad de elementos mutuamente excluyentes.
- **NotNullConverter:** Devuelve true si el valor no es nulo. Se usa para la visibilidad condicional de elementos.
- **NotificacionColorConverter:** Devuelve el color de fondo de una tarjeta de notificación según su tipo (peligro, advertencia, cumplido, fallido).
- **PeriodoObjetivoToStringConverter:** Convierte el enum PeriodoObjetivo a su representación en cadena de texto.
- **PosicionRankingConverter:** Convierte la posición numérica en el ranking a un emoji de medalla para los tres primeros puestos o al número ordinal para el resto.
- **TipoSelectorColorConverter:** Gestiona el color del selector de tipo en el formulario de movimientos.

- **ExcedidoColorConverter:** Devuelve un color diferente cuando se ha excedido un límite.

### Carpeta Extensions

- **TaskExtensions:** Nos permite ejecutar tareas sin esperar a que terminen, pero capturando cualquier excepción que pueda ocurrir para evitar que el programa se bloquee.

### Carpeta Resources

Aquí lo que importa son los archivos Colors y Styles, en los cuales definimos la paleta de colores de la aplicación y los estilos reutilizables para los controles principales.

### Carpeta Services

Contiene los servicios que usa la UI para comunicarse con el exterior.

- **IApiService / ApiService:** Es el cliente HTTP que centraliza todas las llamadas a la API REST. Gestiona el token JWT añadiéndolo automáticamente a cada petición en la cabecera Authorization. En caso de error de red o respuesta no exitosa lanza excepciones con el mensaje del servidor para que los ViewModels puedan mostrarlas al usuario.
- **ISessionService / SessionService:** Gestiona la sesión del usuario usando las preferencias del dispositivo. Guarda y recupera el token JWT, el ID, el nombre y el email del usuario. Permite comprobar si hay una sesión activa al arrancar la app.
- **INavigationService / NavigationService:** Centraliza la navegación entre páginas. Abstrae el uso directo de Shell.Current.GoToAsync para facilitar el mantenimiento y evitar cadenas de rutas dispersas por los ViewModels.

### Carpeta ViewModels

Cada view corresponde a una de las pantallas/views de la aplicación. Todas heredan de BaseViewModel, que proporciona una serie de propiedades comunes para todas. Tenemos las siguientes: AjustesViewModel, BaseViewModel, BuscarMovimientosViewModel, CategoríasViewModel, CrearCategoríaViewModel, CrearMovimientoViewModel, CrearObjetivoViewModel, DashboardViewModel, DetalleCategoríaViewModel, DetalleGrupoViewModel, DetalleMovimientoViewModel, DetalleObjetivoViewModel, GráficasViewModel, GruposViewModel, LoginViewModel, MovimientosViewModel, NotificacionesViewModel, ObjetivosViewModel, RegistroViewModel y SolicitudesGrupoViewModel

### Carpeta Views

Son las ContentPage que son cada una de las siguientes pantallas de la aplicación. Tenemos: AjustesPage, BasePage, BuscarMovimientosPage, CategoríasPage, CrearCategoríaPage, CrearMovimientoPage, CrearObjetivoPage, DashboardPage, DetalleCategoríaPage, DetalleGrupoPage, DetalleMovimientoPage, DetalleObjetivoPage, GráficasPage, GruposPage, LoginPage, MovimientosPage, NotificacionesPage, ObjetivosPage, RegistroPage y SolicitudesGrupoPage

### Archivos raíz del proyecto

- **App.xaml.cs:** Es el punto de entrada de la aplicación. Lee el tema guardado en las preferencias y lo aplica antes de mostrar ninguna pantalla, garantizando que tanto el login como el dashboard arrancan con el tema correcto.
- **AppShell.xaml:** Define la estructura de navegación principal. Combina un menú lateral (Flyout) con pestañas inferiores (TabBar). Las pestañas incluyen Inicio, Movimientos, Categorías y Objetivos. El menú lateral incluye Gráficas, Ajustes, Grupos y Notificaciones.
- **MauiProgram.cs:** Configura todos los servicios de la aplicación: registra los ViewModels, las páginas, los servicios y los paquetes de terceros mediante el patrón builder de MAUI.

# Base de Datos

La base de datos de nuestra aplicación está alojada en un servidor de Aiven (MySQL 8) y accedemos a ella a través de la API mediante Entity Framework Core con el proveedor Pomelo.

## Descripción de las Tablas

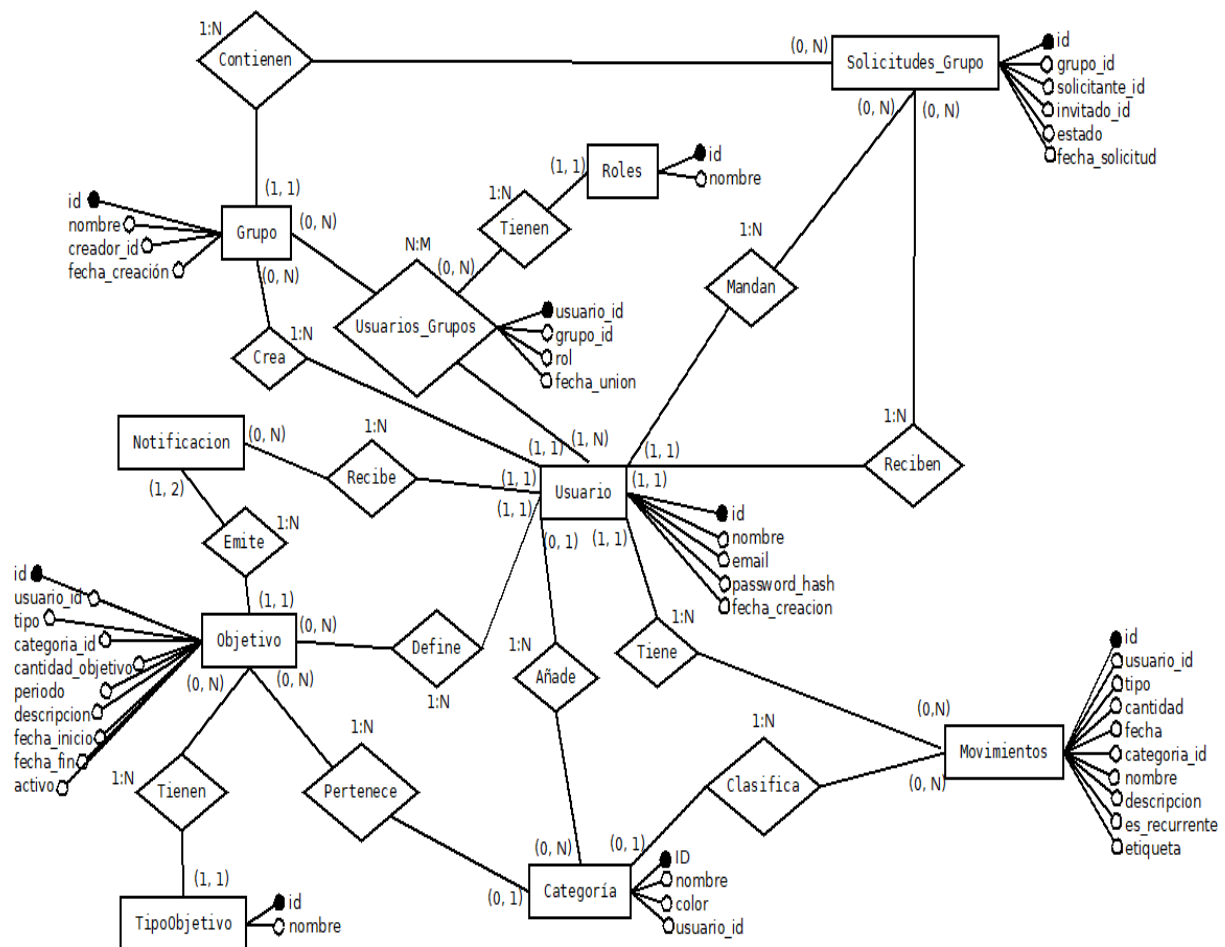
- **Usuario:** Almacena los datos de cada una de las cuentas que se registran en la aplicación.
- **Categorías:** Son las categorías personalizadas de cada usuario para clasificar sus movimientos.
- **Movimientos:** Es la tabla principal para las transacciones económicas.
- **TipoObjetivo:** Almacena los diferentes objetivos que da de opción el programa. Se carga con datos iniciales al crear la base de datos.
- **Objetivos:** Son los objetivos financieros de cada usuario.
- **Grupos:** Grupos de usuarios para compartir información financiera.
- **Roles:** Es el catálogo de roles: administrador y miembro. Se carga con datos iniciales.
- **Usuarios\_Grupos:** Es la tabla de unión entre las entidades de grupo y usuario
- **Solicitudes\_Grupo:** Gestiona el ciclo de invitaciones. La solicitud puede estar pendiente, aceptada o rechazada.
- **Notificaciones:** Se encarga de almacenar las notificaciones persistentes generadas cuando un objetivo se cumple o vence. Las notificaciones de alerta dinámica (peligro, proximidad de vencimiento, exceso de gasto) no se almacenan y se calculan en cada consulta.

## Relaciones entre las Entidades

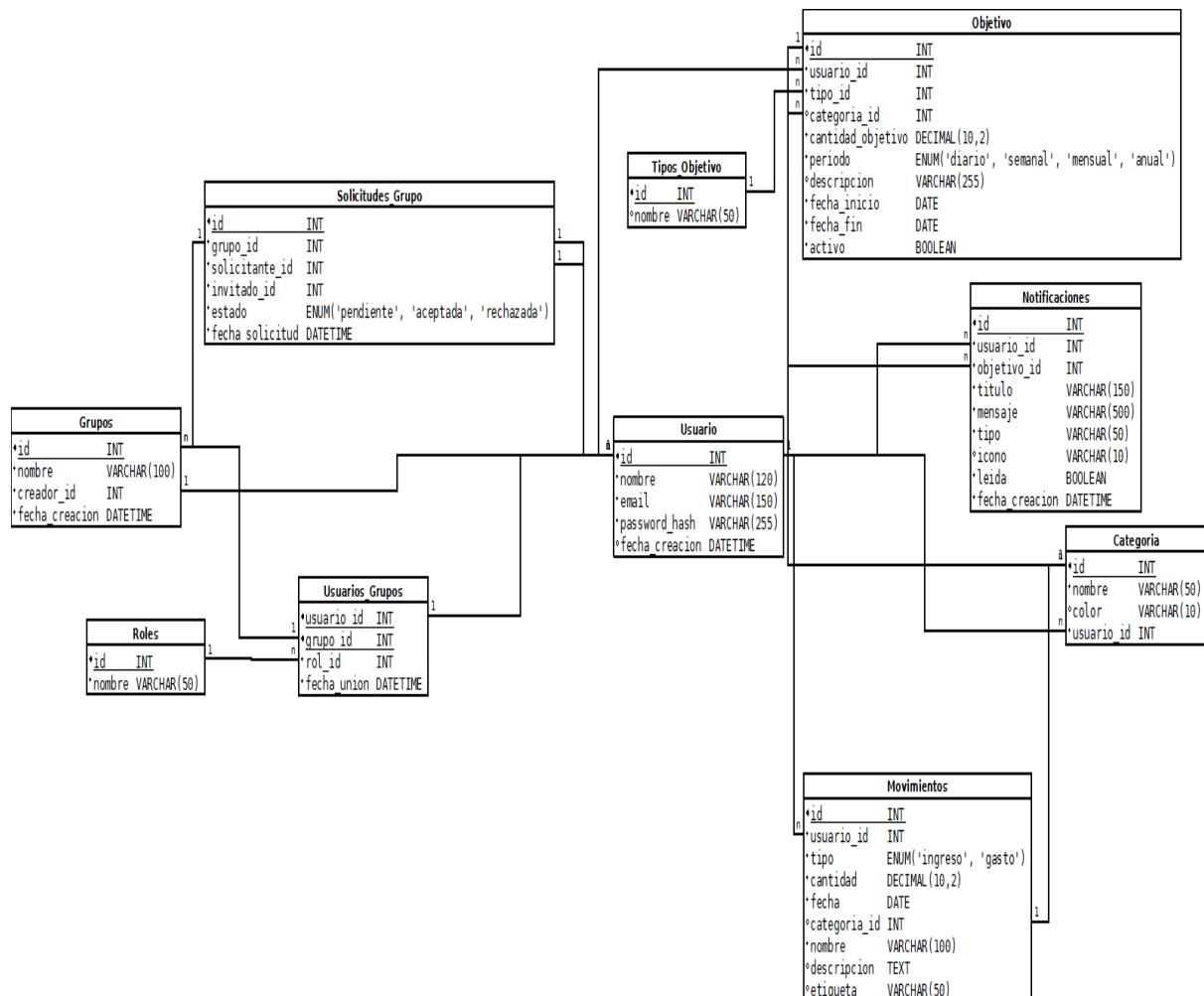
- **Usuario — Define — Objetivo:** Un usuario puede definir 0 o muchos objetivos, mientras que los objetivos tienen que ser definidos por un usuario.
- **Usuario — Añade — Categorías:** Un usuario puede añadir 0 o muchas categorías, mientras que las categorías pueden ser añadidas por un usuario o por ninguno.
- **Usuario — Tiene — Movimientos:** Un usuario puede tener 0 o muchos movimientos, mientras que los movimientos tienen que ser definidos por un usuario.
- **Usuario — Mandan — Solicitudes\_Grupo:** Un usuario puede mandar 0 o muchas solicitudes de grupo, mientras que las solicitudes de grupo pueden ser mandadas por un usuario.
- **Usuario — Reciben — Solicitudes\_Grupo:** Un usuario puede recibir 0 o muchas solicitudes de grupo, mientras que las solicitudes de grupo pueden ser recibidas por un usuario.
- **Usuario — Crea — Grupo:** Un usuario puede crear 0 o muchos grupos, mientras que un grupo es creado por un usuario

- **Usuario — Usuario\_Grupos — Grupo:** Debido a que la relación es N:M para unir usuario y grupos, creamos una tabla intermedia que guarde los usuarios de los grupos
- **Objetivo — Tienen — TipoObjetivo:** Un objetivo tiene un tipo de objetivo, mientras que un tipo de objetivo puede tener 0 o muchos objetivos.
- **Objetivo — Pertenece — Categorías:** Un objetivo pertenece a una categoría o a ninguna, mientras que a una categoría pueden pertenecer, muchos o ningún objetivo.
- **Categoría — Clasifica — Movimientos:** Una categoría clasifica 0 o muchos movimientos, mientras que los movimientos son clasificados en 0 o en una categoría
- **Usuario\_Grupos — Tienen — Roles:** Los usuarios de los grupos tienen un rol, mientras los roles pueden tener 0 o muchos usuarios de grupo.
- **Grupo — Contienen — Solicitudes\_Grupo:** Los grupos contienen 0 o muchas solicitudes de grupo, mientras que las solicitudes de grupo contienen un grupo
- **Objetivo — Emite — Notificaciones:** Los objetivos emiten 1 o 2 notificaciones, mientras que las notificaciones son emitidas por un objetivo.
- **Usuario — Recibe — Notificaciones:** Los usuarios reciben 0 o muchas notificaciones, mientras que las notificaciones son recibidas por un usuario

## Diagrama E/R



## Modelo Relacional



## Normalización

La normalización es el proceso de organizar los atributos de las tablas para reducir la redundancia de los datos y mejorar la integridad.

### 1º Forma Normal

Una tabla está en 1ºFN si:

- Todos los atributos contienen valores atómicos (es decir, indivisibles).
- No hay grupos repetitivos, ni atributos multivaluados.
- Existe una clave primaria que identifica de forma única cada fila.

### Análisis de cada Tabla

- ❖ **Usuarios**: Todos los atributos son atómicos y tiene clave primaria id
- ❖ **Categorías**: Todos los atributos son atómicos y tiene clave primaria id
- ❖ **Movimientos**: Todos los atributos son atómicos y tiene clave primaria id

- ❖ **TipoObjetivo:** Es una tabla simple con dos atributos atómicos y tiene clave primaria id
- ❖ **Objetivos:** Todos los atributos son atómicos y tiene clave primaria id
- ❖ **Grupos:** Todos los atributos son atómicos y tiene clave primaria id
- ❖ **Roles:** Es una tabla simple con dos atributos atómicos y tiene clave primaria id
- ❖ **Usuarios\_Grupos:** Todos los atributos son atómicos y tiene clave primaria id
- ❖ **Solicitudes\_Grupo:** Todos los atributos son atómicos y tiene clave primaria id
- ❖ **Notificaciones:** Todos los atributos son atómicos y tiene clave primaria id

**Conclusión:** Todas las tablas están en 1FN porque cumplen las condiciones.

## Segunda Forma Normal

Una tabla está en 2FN si está en 1FN y además todos los atributos no clave dependen funcionalmente de la clave primaria completa (no de una parte de ella). Esto solo aplica a las tablas con clave primaria completas.

La única tabla que tenemos con clave primaria compuesta es **Usuarios\_Grupos**:

- ❖ **rol\_id:** En el mismo grupo cada usuario puede tener un rol diferente. Por tanto depende de usuario\_id y grupo\_id.
- ❖ **fecha\_union:** Depende de cuando un usuario concreto se unió a un grupo concreto. Por tanto depende de usuario\_id y grupo\_id.

Todos los atributos de **Usuarios\_Grupos** depende de las dos claves primarias (la clave primaria compuesta).

El resto de tablas, solo tienen una clave primaria simple, por lo cual se cumple el requisito para que sean de 2FN.

**Conclusión:** Todas las tablas están en 2FN porque cumplen las condiciones.

## Tercera Forma Normal

Una tabla está en 3FN si está en 2FN y además no existe ninguna dependencia funcional transitiva entre atributos no clave. Es decir, ningún atributo no clave debe depender de otro atributo no clave

### Análisis de cada Tabla

- ❖ **Usuarios:** nombre, email, password\_hash y fecha\_creacion dependen directamente de id. No hay ningún atributo que dependa de otro atributo no clave.
- ❖ **Categorías:** nombre, color y usuario\_id dependen directamente de id. No hay ningún atributo que dependa de otro atributo no clave.
- ❖ **Movimientos:** usuario\_id, tipo, cantidad, fecha, categoria\_id, nombre, descripcion y etiqueta dependen directamente de id. No hay ningún atributo que dependa de otro atributo no clave.
- ❖ **TipoObjetivo:** nombre depende directamente de id. No hay ningún atributo que dependa de otro atributo no clave.
- ❖ **Objetivos:** usuario\_id, tipo\_id, categoria\_id, cantidad\_objetivo, periodo, descripcion, fecha\_inicio, fecha\_fin y activo. No hay ningún atributo que dependa de otro atributo no clave.
- ❖ **Grupos:** nombre, creador\_id y fecha\_creacion dependen directamente de id. No hay ningún atributo que dependa de otro atributo no clave.

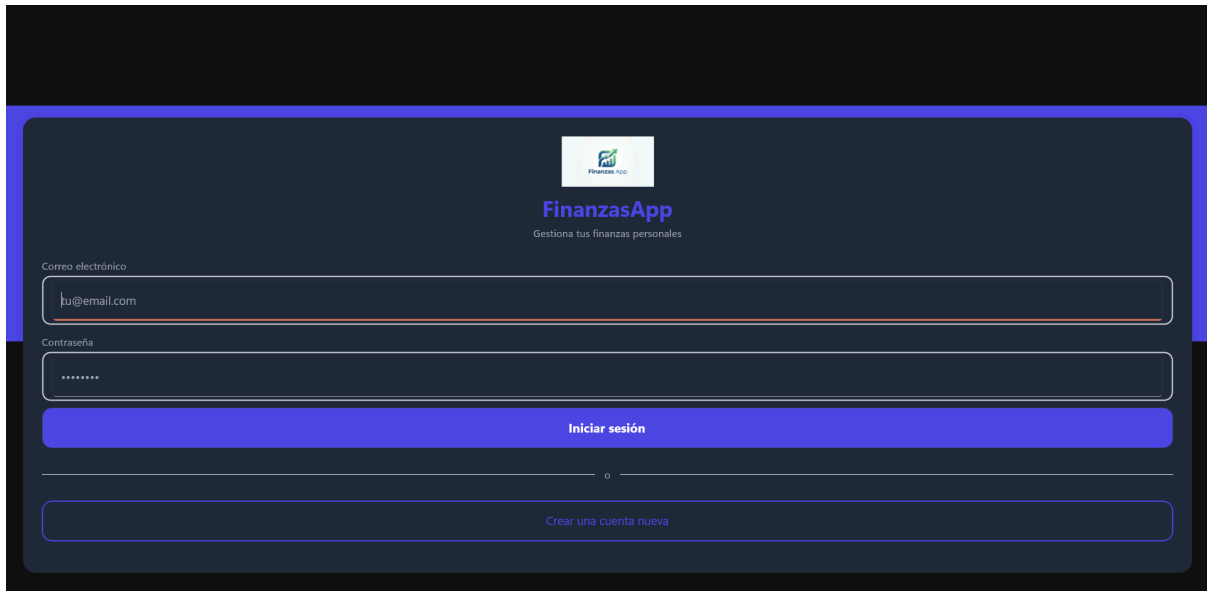
- ❖ **Roles:** nombre depende directamente de id. No hay ningún atributo que dependa de otro atributo no clave.
- ❖ **Usuarios\_Grupos:** rol\_id y fecha\_union dependen directamente de usuario\_id y grupo\_id. No hay ningún atributo que dependa de otro atributo no clave.
- ❖ **Solicitudes\_Grupo:** grupo\_id, solicitante\_id, invitado\_id, estado y fecha\_solicitud. No hay ningún atributo que dependa de otro atributo no clave.
- ❖ **Notificaciones:** usuario\_id, objetivo\_id, titulo, mensaje, tipo, icono, leida y fecha\_creacion. No hay ningún atributo que dependa de otro atributo no clave.

**Conclusión:** Todas las tablas están en 3FN. No tenemos redundancias derivadas de dependencias transitivas. Cada dato se almacena en un único lugar y se referencian mediante claves foráneas cuando es necesario .

# Manual de Usuario

## Registro e Inicio de Sesión

Al abrir la aplicación la primera vez o cuando se agota la sesión. Si no tienes cuenta deberás de darle a “Crear una nueva Cuenta”



El formulario de registro pide tres datos:

- **Nombre completo:** El nombre con el que la aplicación conocerá al usuario.
- **Correo electrónico:** Se usa como identificador único para entrar en la aplicación. Debe ser un email válido ya que no puede haber dos cuentas con el mismo correo.
- **Contraseña:** Debe tener al menos ocho caracteres. Se pide dos veces para confirmar que no hay errores al escribirla.

Si los datos son correctos al darle a crear, la cuenta se creará sin problema y te redirigirá al dashboard. Y podrás iniciar sesión con tu nueva cuenta.

**Realizado por: Roberto Gómez Suárez**

## Navegación por la aplicación

La aplicación tiene dos formas para desplazarse a las diferentes secciones de la aplicación.

### Menú Principal

En Windows aparece arriba, en android aparece abajo. Tiene 4 secciones: Inicio, Movimientos, Categorías y Objetivos



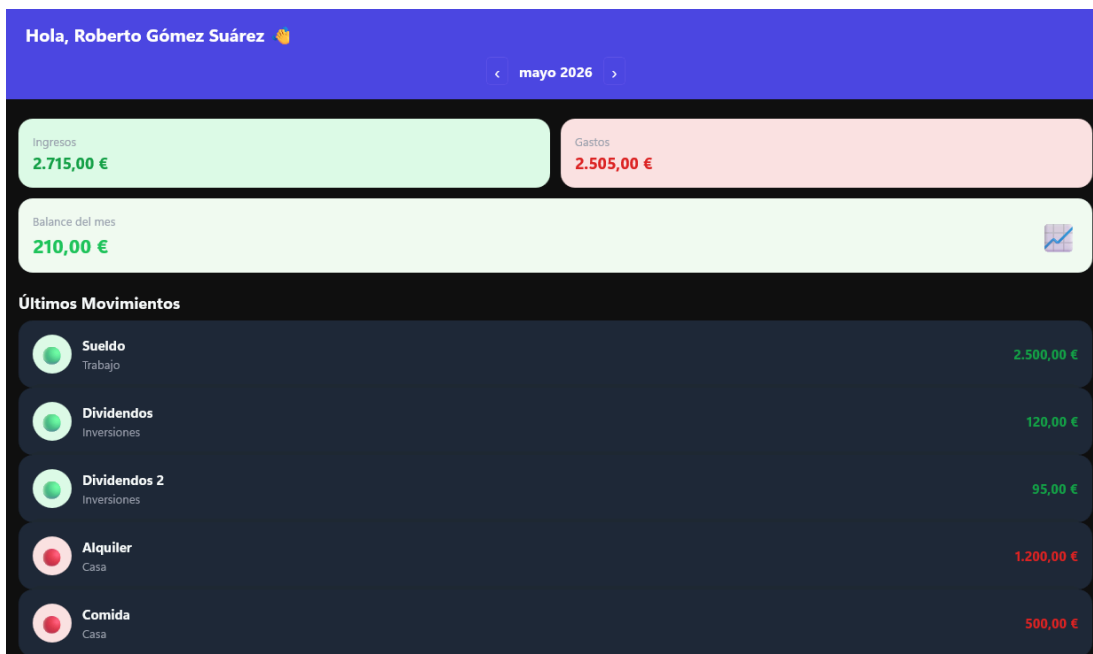
### Menú lateral

Deslizando desde el borde izquierdo de la pantalla en android, o pulsando las 3 barras desde Windows. Tiene 4 secciones: Gráficas, Ajustes, Grupos y Notificaciones



## Dashboard (Inicio)

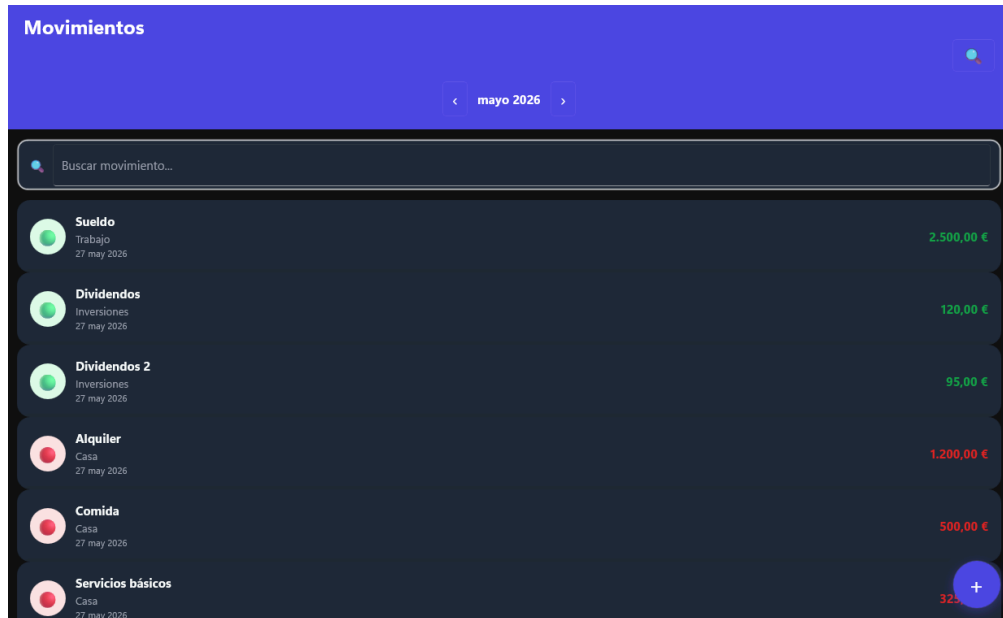
La pantalla de inicio muestra un resumen financiero del mes seleccionado. Presenta tres tarjetas: ingresos totales del mes, gastos totales del mes y balance (diferencia entre ingresos y gastos). También muestra los cinco últimos movimientos registrados para tener una visión rápida de las actividades recientes.



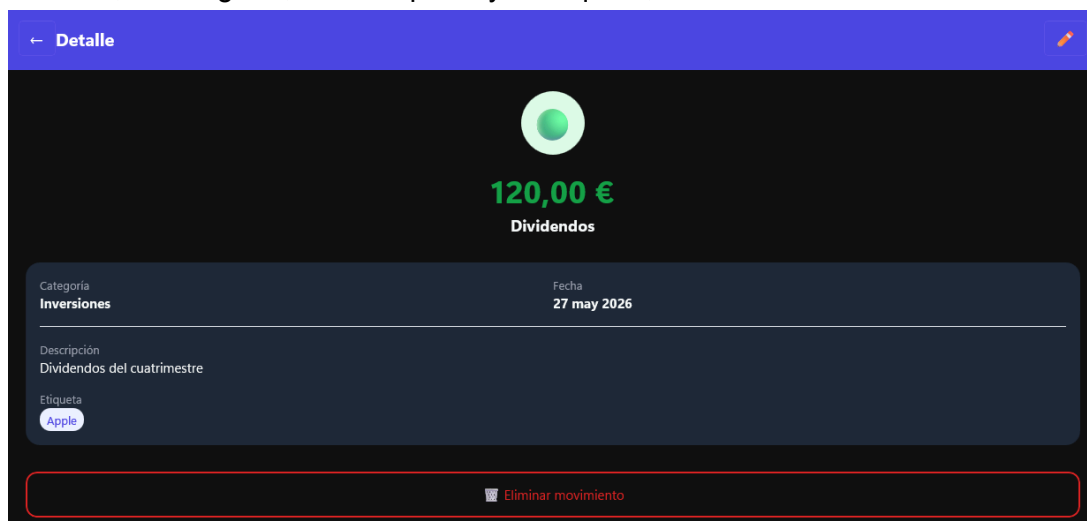
El usuario puede navegar entre meses usando los botones de anterior y siguiente. El dashboard también se puede actualizar por el deslizamiento hacia abajo.

## Movimientos

En la pantalla de movimientos se listan todos los movimientos del mes actual ordenados por fecha descendente. Hay un buscador que filtra en tiempo real por nombre, permitiendo localizar rápidamente cualquier transacción. El usuario puede navegar entre meses igual que en el dashboard.



Al pulsar sobre un movimiento se accede al detalle, donde se pueden ver todos sus campos. Desde el detalle es posible activar el modo edición para modificar el nombre, la cantidad, la categoría, la descripción y la etiqueta, o eliminar el movimiento.



El botón de crear (el botón flotante con el símbolo +) abre el formulario de nuevo movimiento. En este formulario se elige si es un ingreso o un gasto mediante un selector, se introduce el nombre y la cantidad (campos obligatorios), y opcionalmente se asigna una categoría, una descripción y una etiqueta.

The screenshot shows a form titled "Nuevo Movimiento" with a blue header bar. The form fields are as follows:

- Tipo:** A horizontal bar with a red segment on the left and a green segment on the right labeled "Ingreso".
- Nombre \*:** A text input field containing "Ej: Nómina, Supermercado...".
- Cantidad \*:** A text input field containing "0.00".
- Categoría:** A dropdown menu with the placeholder text "Selecciona una categoría".
- Descripción:** A large text area with the placeholder text "Añade una nota opcional...".
- Etiqueta:** A text input field containing "Ej: trabajo, ocio...".
- Guardar movimiento:** A blue button at the bottom.

Desde la pantalla de movimientos también se puede acceder a la búsqueda avanzada, que permite filtrar por texto libre y por tipo (todos, ingresos o gastos) con indicación del mes actual.

The screenshot shows a search screen titled "Buscar Movimientos" with a blue header bar. The search results are as follows:

- Search Bar:** Contains the text "Apple" with a magnifying glass icon on the left and "x" and "x" icons on the right.
- Filters:** A row of buttons: "mayo 2026" (blue), "Todos" (white), "Ingresos" (green), and "Gastos" (red).
- Result:** A card for "Dividendos" (Inversiones) with a green icon, the date "27 may 2026", and the amount "120,00 €" in green.

## Categorías

Las categorías sirven para clasificar los movimientos. En la lista se muestran todas las categorías del usuario con su color identificativo. Desde la lista se puede eliminar o crear una nueva.

The screenshot shows a list titled "Categorías" with a blue header bar. The list contains five categories, each with a colored icon, a name, and a trash icon on the right:

- Casa:** Red icon.
- Inversiones:** Orange icon.
- Ocio:** Pink icon.
- Trabajo:** Green icon.
- Transporte:** Blue icon.

Al crear una categoría se introduce el nombre y se elige el color entre una paleta de colores. Una vista previa muestra en tiempo real cómo quedará el icono de la categoría con el color elegido.

La interfaz muestra una barra superior azul con el título '← Nueva Categoría'. Debajo, hay un icono de una etiqueta amarilla en un círculo azul. Sección 'Nombre': un campo de texto con el placeholder 'Introduzca el Nombre de la nueva Categoría'. Sección 'Color': una fila de diez círculos de colores (azul, verde, rojo, naranja, púrpura, cian, magenta, lima, naranja, gris). En la parte inferior hay un botón azul que dice 'Crear Categoría'.

Al pulsar sobre una categoría se accede a su detalle, donde se listan los últimos movimientos del mes que tienen esa categoría asignada.

La interfaz muestra una barra superior azul con el título '← Casa' y el subtítulo 'Últimos 10 movimientos del mes'. La lista de movimientos es la siguiente:

|  |                                         |            |
|--|-----------------------------------------|------------|
|  | <b>Alquiler</b><br>27 may 2026          | 1.200,00 € |
|  | <b>Comida</b><br>27 may 2026            | 500,00 €   |
|  | <b>Servicios básicos</b><br>27 may 2026 | 325,00 €   |

## Objetivos

Los objetivos permiten establecer metas financieras con un seguimiento.

La interfaz muestra una barra superior azul con el título 'Objetivos'. En el centro hay un icono de un blanco y una flecha roja, con el texto 'No tienes objetivos aún' debajo. En la esquina inferior derecha hay un botón circular azul con un signo '+'. En la parte inferior de la pantalla, se ve la parte superior de un formulario para crear un objetivo.

Hay cuatro tipos disponibles:

- **Ahorro:** Controla que el balance (ingresos menos gastos) del periodo supere una cantidad determinada. No filtra por categoría ya que el ahorro es un concepto global, no de una sola categoría

El formulario tiene un título 'Tipo de objetivo'. Debajo hay un campo de selección con el placeholder 'Selecciona un tipo' y la opción 'Ahorro' seleccionada. En la parte inferior, un recuadro azul claro contiene el texto: 'El ahorro mide tu balance (ingresos - gastos) en el periodo. No filtra por categoría'.

- **Límite de Gasto:** Controla que los gastos del periodo no superen un límite establecido. La categoría es opcional: puede limitarse el gasto total o solo el de una categoría concreta

Tipo de objetivo

Selecciona un tipo

Límite de gasto

Controla que tus gastos no superen el límite en el periodo. Puedes limitarlo a una categoría concreta o a todos tus gastos.

- **Meta de Ingreso:** Controla que los ingresos totales del periodo alcancen una cantidad objetivo

Tipo de objetivo

Selecciona un tipo

Meta de ingreso

Controla que tus ingresos totales alcancen la cantidad objetivo en el periodo.

- **Reducción de Deuda:** Controla que los gastos de una categoría concreta se mantengan por debajo de un límite establecido. La categoría es obligatoria en este tipo.

Tipo de objetivo

Selecciona un tipo

Reducción de deuda

Controla que los gastos de una categoría concreta se mantengan por debajo del límite. La categoría es obligatoria.

En la pantalla de objetivos se lista cada objetivo con una barra de progreso visual, el texto de progreso con las cantidades actuales frente al objetivo, las fechas de inicio y fin, y un badge que indica el estado actual. Si el objetivo ha vencido se muestra una advertencia.

### Objetivos

#### Llegar a ganar 3.000 €

En progreso

Meta de ingreso

Ingresado \$2,715.00 de \$3,000.00

26/05/26 → 28/05/26

#### No gastar mucho en Ocio

Límite superado

Límite de gasto

Gastado \$340.00 de \$100.00 límite

26/05/26 → 28/06/26

El formulario de creación y edición adapta los campos que muestra según el tipo de objetivo seleccionado, ocultando el selector de categoría cuando no es aplicable y mostrando un texto de ayuda contextual que explica qué mide ese tipo de objetivo.

← Detalle objetivo



Descripción

Llegar a ganar 3.000 €

Tipo

Meta de ingreso

Periodo

Mensual

Categoría

Sin categoría

Cantidad objetivo

3.000,00 €

Fecha inicio

26/05/2026

Fecha fin

28/05/2026

Eliminar objetivo

← Nuevo objetivo

Tipo de objetivo

Selecciona un tipo

Ahorro

El ahorro mide tu balance (ingresos - gastos) en el periodo. No filtra por categoría

Descripción (Opcional)

Introduzca la descripción

Cantidad Objetivo

0,00

Periodo del objetivo

Fecha inicio

27/05/2026

Fecha fin

27/06/2026

Crear objetivo

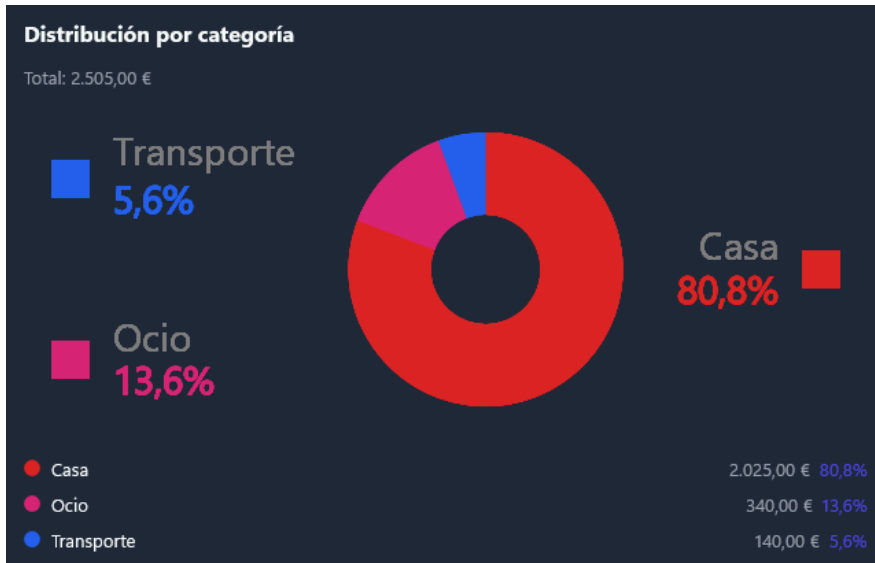
El progreso de cada objetivo se calcula en tiempo real por el servidor en cada consulta, sumando los movimientos del usuario que caen dentro del rango de fechas definido.

## Gráficas

La pantalla de gráficas ofrece dos visualizaciones del comportamiento financiero del usuario.

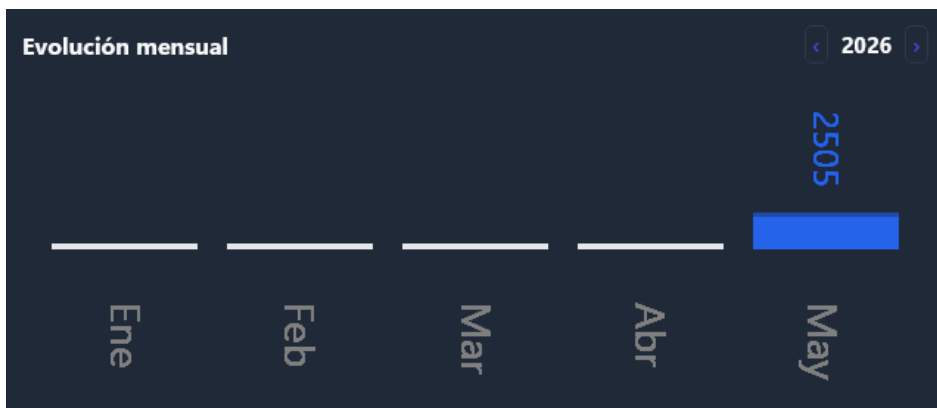
La **gráfica de distribución por categoría** es una gráfica de tipo donut que muestra qué porcentaje del total de gastos o ingresos corresponde a cada categoría del mes

seleccionado. Debajo del gráfico hay una leyenda con el nombre, el importe y el porcentaje de cada categoría.

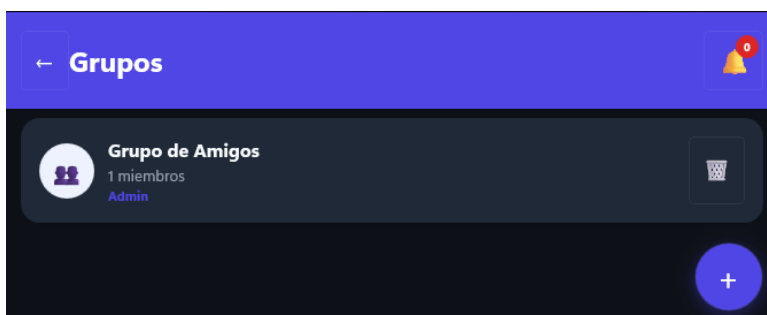


La **gráfica de evolución mensual** es una gráfica de barras que muestra los doce meses del año seleccionado con el total de gastos o ingresos de cada mes. Permite navegar entre años para ver la evolución histórica.

El usuario puede alternar entre ver los datos de gastos o de ingresos mediante dos botones en la cabecera.

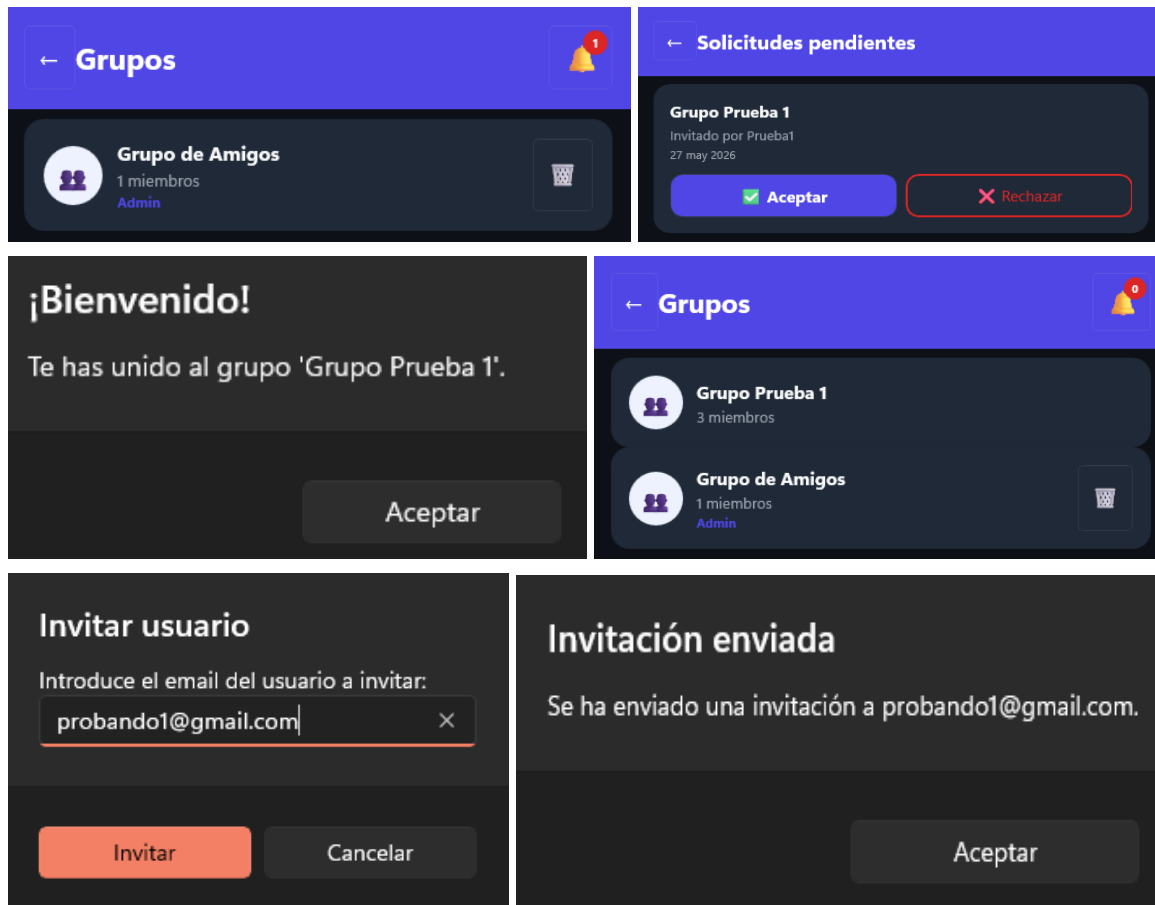


## Grupos



Los grupos permiten que varios usuarios compartan un espacio común donde ver el resumen financiero consolidado y comparar sus gastos del mes mediante un ranking.

Cualquier usuario puede crear un grupo, convirtiéndose automáticamente en su administrador. Para añadir miembros se introduce el email del usuario a invitar. El usuario invitado recibirá una solicitud pendiente que puede aceptar o rechazar desde la pantalla de solicitudes, accesible desde el icono de campana en la cabecera de grupos.



Al entrar en el detalle de un grupo se muestra el resumen financiero consolidado del mes (suma de todos los gastos e ingresos de los miembros), el balance del grupo y el ranking de gastos ordenado de mayor a menor, con medallas de oro, plata y bronce para los tres primeros puestos. Solo el creador del grupo puede eliminarlo. La eliminación es definitiva y borra todos los datos relacionados.



## Notificaciones

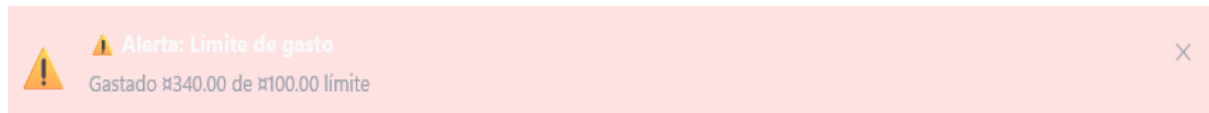


El sistema de notificaciones es automático y no requiere configuración por parte del usuario. Hay dos tipos:

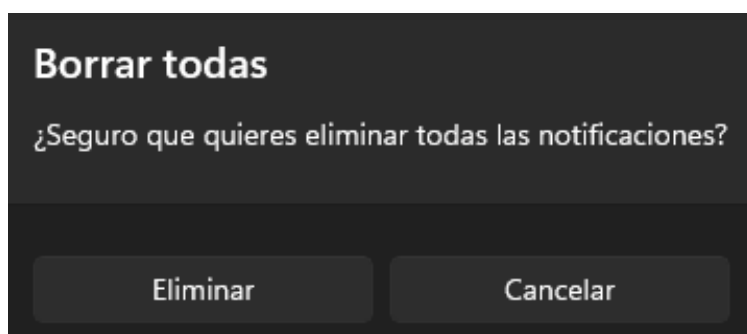
Las **notificaciones persistentes** se generan cuando un objetivo se cumple o cuando vence sin haberse cumplido. Se guardan en la base de datos y permanecen hasta que el usuario las elimina. Una vez que un objetivo genera una notificación persistente, se marca como inactivo para que no vuelva a evaluarse.



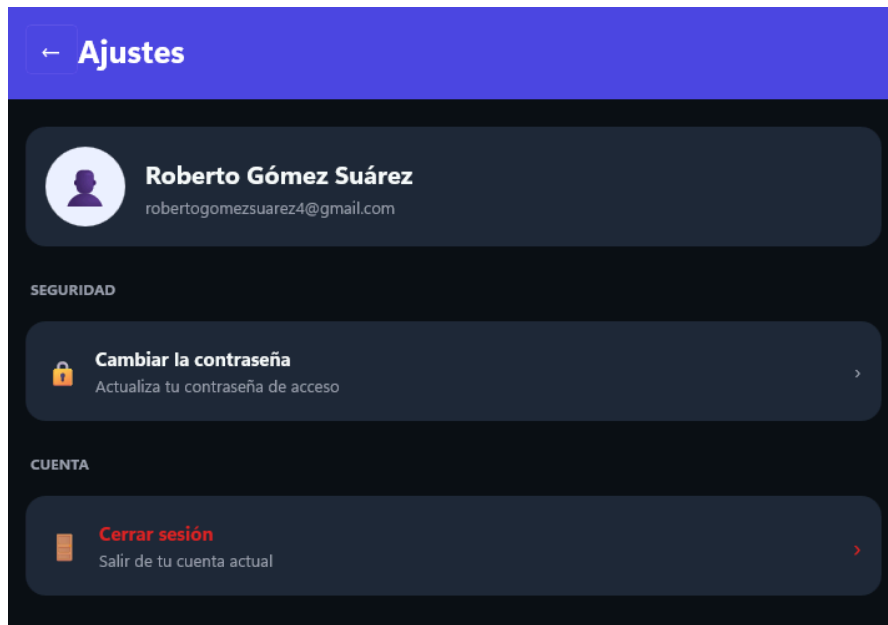
Las **notificaciones dinámicas** se calculan en cada consulta y no se guardan. Incluyen alertas cuando un objetivo está en peligro (por ejemplo al llegar al 80% del límite de gasto), avisos de vencimiento próximo con el tiempo exacto restante en horas y minutos, y un aviso cuando el gasto del mes supera en más de un 20% el del mes anterior.



El usuario puede eliminar cualquier notificación individualmente o todas a la vez con el botón del contenedor de basura en la cabecera.



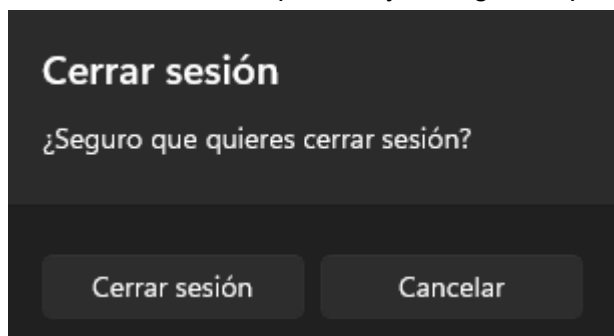
## Ajustes



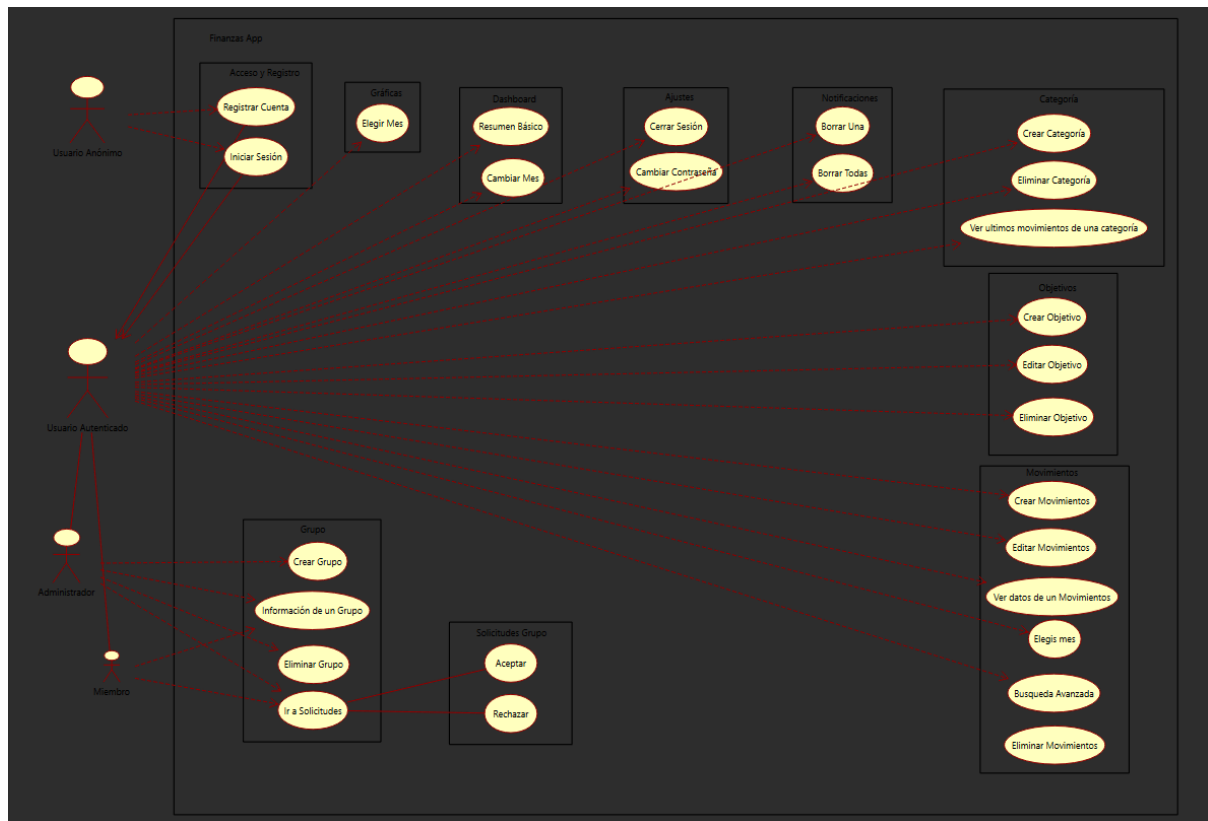
En la pantalla de ajustes el usuario puede realizar dos acciones:

- **Cambio de contraseña:** Mediante un formulario desplegable que solicita la contraseña actual, la nueva contraseña y su confirmación. Se valida que la nueva contraseña tenga al menos ocho caracteres y sea diferente a la actual

- **Cierre de sesión:** Previa confirmación mediante una alerta, elimina los datos de sesión del dispositivo y redirige a la pantalla de login



## Diagrama de Casos de Uso



Primero eres un usuario anónimo, una vez que inicias sesión o te registras, pasas a ser un usuario autenticado, y a partir de ahí puedes acceder a las funciones dentro de cada sección del programa: Dashboard (Resumen Básico y Cambiar Mes), Movimientos (Crear, Editar, Eliminar, Ver datos de un Movimiento, elegir mes y búsqueda avanzada), Categoría (Crear, Eliminar y Ver los últimos movimientos de una categoría), Objetivo (Crear, Editar Y Eliminar Objetivo), Gráficas (Elegir mes), Ajustes (Cerrar Sesión y Cambiar Contraseña), Notificaciones (Borrar Una y Borrar Todas) y Grupos (Crear y Eliminar un Grupo, Ver la Información de un Grupo e ir a las solicitudes).

Dentro de los grupos hay roles Administrador y Miembro (Un usuario puede ser Miembro en un grupo y administrador en otros), el administrador tiene acceso a todas las funciones de la sección y el miembro, no puede crear un grupo, porque se convertiría en administrador de ese grupo, y tampoco puede borrar un grupo siendo miembro del mismo.

# Pruebas

## Pruebas de la Interfaz

| ID    | Caso de Prueba                       | Datos de Entrada                                | Resultado Esperado                                                  | Resultado Obtenido |
|-------|--------------------------------------|-------------------------------------------------|---------------------------------------------------------------------|--------------------|
| PR1_1 | Abrir la app sin sesión activa       | Sin token guardado en el dispositivo            | Se muestra la pantalla de Login                                     | Correcto           |
| PR1_2 | Abrir la app con sesión activa       | Token válido guardado en Preferences            | Se navega directamente al Dashboard sin pasar por Login             | Correcto           |
| PR1_3 | Login con campos vacíos              | Email: vacío,<br>Password: vacío                | Mensaje de error "Por favor, completa todos los campos"             | Error Controlado   |
| PR1_4 | Login con email sin formato válido   | Email: noesuncorreo,<br>Password: 12345678      | Mensaje de error "El correo electrónico no tiene un formato válido" | Error Controlado   |
| PR1_5 | Login con credenciales incorrectas   | Email: test@test.com,<br>Password: passwordmalo | Mensaje de error "Credenciales incorrectas" devuelto por la API     | Error Controlado   |
| PR1_6 | Login correcto                       | Email y password válidos de cuenta existente    | Se navega al Dashboard y se muestra el nombre del usuario           | Correcto           |
| PR1_7 | Registro con todos los campos vacíos | Nombre, Email y Password vacíos                 | Mensaje de error "Por favor, completa todos los campos"             | Error Controlado   |

## Pruebas

|        |                                               |                                          |                                                                      |                  |
|--------|-----------------------------------------------|------------------------------------------|----------------------------------------------------------------------|------------------|
| PR1_8  | Registro con contraseña menor de 8 caracteres | Password: 1234                           | Mensaje de error "La contraseña debe tener al menos 8 caracteres"    | Error Controlado |
| PR1_9  | Registro con email ya existente               | Email de una cuenta ya registrada        | Mensaje de error "Ya existe una cuenta con ese email"                | Error Controlado |
| PR1_10 | Registro correcto                             | Todos los campos válidos y email nuevo   | Se crea la cuenta, se navega al Dashboard y se muestra el nombre     | Correcto         |
| PR1_11 | Dashboard carga el resumen del mes actual     | Sesión activa, movimientos registrados   | Se muestran ingresos, gastos y balance del mes actual                | Correcto         |
| PR1_12 | Dashboard muestra los últimos 5 movimientos   | Más de 5 movimientos registrados         | Solo se muestran los 5 más recientes ordenados por fecha descendente | Correcto         |
| PR1_13 | Balance positivo muestra emoji correcto       | Ingresos > Gastos                        | El emoji de la tarjeta de balance es positivo                        | Correcto         |
| PR1_14 | Balance negativo muestra emoji correcto       | Gastos > Ingresos                        | El emoji de la tarjeta de balance es negativo                        | Correcto         |
| PR1_15 | Editar el nombre de un movimiento             | Nombre cambiado en modo edición          | Al guardar, el nombre se actualiza correctamente                     | Correcto         |
| PR1_16 | Editar la cantidad de un movimiento           | Cantidad cambiado en modo edición        | Al guardar, la cantidad se actualiza correctamente                   | Correcto         |
| PR1_17 | Eliminar un movimiento desde el Detalle       | Tap en "Eliminar movimiento" + confirmar | El movimiento se elimina y se vuelve a la lista                      | Correcto         |

## Pruebas

|        |                                              |                                               |                                                                               |                  |
|--------|----------------------------------------------|-----------------------------------------------|-------------------------------------------------------------------------------|------------------|
| PR1_18 | Crear nuevo movimiento con campos vacíos     | Nombre vacío y cantidad 0                     | Mensaje de error indicando campos obligatorios                                | Error Controlado |
| PR1_19 | Crear movimiento con cantidad 0              | Nombre: Test, Cantidad: 0                     | Mensaje de error indicando que la cantidad debe ser mayor que 0               | Error Controlado |
| PR1_20 | Crear movimiento con cantidad negativa       | Cantidad: -50                                 | Mensaje de error indicando cantidad no válida                                 | Error Controlado |
| PR1_21 | Crear movimiento de tipo Gasto               | Tipo: Gasto, Nombre: Compra, Cantidad: 50     | El movimiento se crea y aparece en rojo en la lista                           | Correcto         |
| PR1_22 | Crear movimiento de tipo Ingreso             | Tipo: Ingreso, Nombre: Nómina, Cantidad: 1500 | El movimiento se crea y aparece en verde en la lista                          | Correcto         |
| PR1_23 | Crear categoría con nombre vacío             | Nombre vacío                                  | Mensaje de error "El nombre es obligatorio"                                   | Error Controlado |
| PR1_24 | Eliminar categoría con movimientos asignados | Categoría que tiene movimientos               | La categoría se elimina y los movimientos quedan sin categoría (no se borran) | Correcto         |
| PR1_25 | Barra de progreso de objetivo en peligro     | Objetivo con EstaEnPeligro=true               | La barra de progreso se muestra en rojo                                       | Correcto         |
| PR1_26 | Badge de objetivo cumplido                   | Objetivo con SeHaCumplido=true                | El badge muestra el estado en color verde                                     | Correcto         |
| PR1_27 | Crear objetivo con cantidad 0                | Cantidad: 0                                   | Mensaje de error "Introduce una cantidad válida mayor que 0"                  | Error Controlado |

## Pruebas

|        |                                                      |                                                                 |                                                                             |                  |
|--------|------------------------------------------------------|-----------------------------------------------------------------|-----------------------------------------------------------------------------|------------------|
| PR1_28 | Crear objetivo con fecha fin anterior a inicio       | FechaFin < FechaInicio                                          | Mensaje de error "La fecha fin no puede ser anterior a la fecha de inicio"  | Error Controlado |
| PR1_29 | Crear objetivo tipo Reducción de deuda sin categoría | Tipo: ReduccionDeuda, sin categoría                             | Mensaje de error "La reducción de deuda requiere seleccionar una categoría" | Error Controlado |
| PR1_30 | Eliminar grupo sin ser creador                       | Tap en papelera de grupo ajeno                                  | Mensaje de error "Sólo el creador puede eliminar el grupo"                  | Error Controlado |
| PR1_31 | Invitar a uno mismo                                  | Email del propio usuario autenticado                            | Mensaje de error "No puedes invitarte a ti mismo"                           | Error Controlado |
| PR1_32 | Invitar a usuario ya miembro del grupo               | Email de alguien que ya está en el grupo                        | Mensaje de error "Este usuario ya es miembro del grupo"                     | Error Controlado |
| PR1_33 | Notificación de peligro de objetivo                  | Objetivo con gastos al 80% del límite                           | Se muestra alerta de peligro con el mensaje correspondiente                 | Correcto         |
| PR1_34 | Notificación de exceso de gasto mensual              | Gasto del mes >20% superior al mes anterior                     | Se muestra aviso de exceso de gasto                                         | Correcto         |
| PR1_35 | Cambiar contraseña correctamente                     | Contraseña actual correcta, nueva válida, confirmación coincide | Mensaje de éxito "Contraseña actualizada correctamente"                     | Correcto         |
| PR1_36 | Notificación persistente no se duplica               | Objetivo ya cumplido con notificación generada                  | No se genera una segunda notificación para el mismo objetivo                | Correcto         |

## Pruebas

|        |                                     |                                             |                                                           |                  |
|--------|-------------------------------------|---------------------------------------------|-----------------------------------------------------------|------------------|
| PR1_37 | Objetivo cumplido se marca inactivo | Objetivo de tipo Ahorro con balance >= meta | El objetivo aparece como inactivo y no vuelve a evaluarse | Correcto         |
| PR1_38 | Sin conexión a internet             | Sin conexión, intento de usar la aplicación | Mensaje de error de red, datos no cargados                | Error Controlado |

## Pruebas de la Base de Datos y la API

| ID    | Caso de Prueba                          | Datos de Entrada                                | Resultado Esperado                                                      | Resultado Obtenido |
|-------|-----------------------------------------|-------------------------------------------------|-------------------------------------------------------------------------|--------------------|
| PR2_1 | Registro de usuario nuevo               | Email único, nombre, password                   | Usuario creado en BD con password hashado, token JWT devuelto           | Correcto           |
| PR2_2 | Registro con email duplicado            | Email ya existente en BD                        | Error 400 "Ya existe una cuenta con ese email"                          | Error Controlado   |
| PR2_3 | Password no se almacena en texto plano  | Registro con password cualquiera                | El campo password_hash contiene el hash BCrypt, nunca el texto original | Correcto           |
| PR2_4 | Login con credenciales incorrectas      | Email y password válidos                        | Token JWT devuelto con claim Sub=usuarioid, expiración 72h              | Correcto           |
| PR2_5 | Login con email inexistente             | Email no registrado                             | Error 401 "Credenciales incorrectas"                                    | Error Controlado   |
| PR2_6 | Login con password incorrecto           | Email correcto, password equivocado             | Error 401 "Credenciales incorrectas"                                    | Error Controlado   |
| PR2_7 | Petición sin token a endpoint protegido | GET /api/movimientos sin cabecera Authorization | Respuesta 401 Unauthorized                                              | Error Controlado   |

## Pruebas

|        |                                                  |                                                          |                                                                                             |                  |
|--------|--------------------------------------------------|----------------------------------------------------------|---------------------------------------------------------------------------------------------|------------------|
| PR2_8  | Petición con token expirado                      | Token con fecha exp pasada                               | Respuesta 401 Unauthorized                                                                  | Error Controlado |
| PR2_9  | Petición con token manipulado                    | Token con firma inválida                                 | Respuesta 401 Unauthorized                                                                  | Error Controlado |
| PR2_10 | Obtener solo categorías del usuario autenticado  | GET /api/categorias con token de usuario A               | Solo se devuelven las categorías de usuario A, nunca las de B                               | Correcto         |
| PR2_11 | Editar categoría ajena                           | PUT /api/categorias/{id} de categoría de otro usuario    | Error 403 Forbidden                                                                         | Error Controlado |
| PR2_12 | Eliminar categoría ajena                         | DELETE /api/categorias/{id} de categoría de otro usuario | Error 403 Forbidden                                                                         | Error Controlado |
| PR2_13 | Eliminar categoría con movimientos asignados     | DELETE sobre categoría con movimientos                   | La categoría se elimina y los movimientos quedan con categoria_id=NULL (ON DELETE SET NULL) | Correcto         |
| PR2_14 | Crear movimiento con cantidad decimal            | Cantidad: 19.99                                          | El movimiento se crea con cantidad 19.99 en BD (DECIMAL 10,2)                               | Correcto         |
| PR2_15 | Crear movimiento sin nombre                      | nombre: vacío                                            | Error 400                                                                                   | Error Controlado |
| PR2_16 | Obtener movimientos solo del usuario autenticado | GET /api/movimientos con token de usuario A              | Solo se devuelven movimientos de usuario A                                                  | Correcto         |
| PR2_17 | Resumen del mes calcula ingresos correctamente   | Movimientos de ingreso en el mes                         | TotalIngresos = suma exacta de todos los importes de ingreso del mes                        | Correcto         |

## Pruebas

|        |                                               |                                              |                                                                   |                  |
|--------|-----------------------------------------------|----------------------------------------------|-------------------------------------------------------------------|------------------|
| PR2_18 | Resumen del mes calcula gastos correctamente  | Movimientos de gasto en el mes               | TotalGastos = suma exacta de todos los importes de gasto del mes  | Correcto         |
| PR2_19 | Editar movimiento ajeno                       | PUT /api/movimientos/{id} de otro usuario    | Error 403 Forbidden                                               | Error Controlado |
| PR2_20 | Eliminar movimiento ajeno                     | DELETE /api/movimientos/{id} de otro usuario | Error 403 Forbidden                                               | Error Controlado |
| PR2_21 | Eliminar usuario elimina sus movimientos      | DELETE en cascada del usuario                | Todos los movimientos del usuario se eliminan (ON DELETE CASCADE) | Correcto         |
| PR2_22 | Crear objetivo con tipold inexistente         | tipold=99                                    | Error de restricción de clave foránea                             | Error Controlado |
| PR2_23 | Evaluar objetivo tipo Ahorro cumplido         | balance del periodo >= cantidadObjetivo      | ObjetivoSeHaCumplido=true, Estado="Meta alcanzada"                | Correcto         |
| PR2_24 | Evaluar objetivo tipo LimiteGasto superado    | gastos > límite al vencer                    | SeHaCumplido=false, estado=Límite superado                        | Correcto         |
| PR2_25 | EvaluarTodos no genera N+1 consultas          | Usuario con 10 objetivos activos             | Los movimientos se cargan una sola vez para todos los objetivos   | Correcto         |
| PR2_26 | Editar objetivo ajeno                         | PUT /api/objetivos/{id} de otro usuario      | Error 403 Forbidden                                               | Error Controlado |
| PR2_27 | EsMiembro devuelve false para no miembro      | Usuario que no pertenece al grupo            | EsMiembro=false, acceso denegado al resumen                       | Error Controlado |
| PR2_28 | Eliminar grupo solo lo puede hacer el creador | DELETE /api/grupos/{id} por no creador       | Error 403 Forbidden                                               | Error Controlado |

## Pruebas

|        |                                                              |                                                          |                                                                       |                  |
|--------|--------------------------------------------------------------|----------------------------------------------------------|-----------------------------------------------------------------------|------------------|
| PR2_29 | Invitar usuario por email inexistente                        | Email no registrado                                      | Error 404 "No existe ningún usuario con ese email"                    | Error Controlado |
| PR2_30 | No se puede invitar a uno mismo                              | Email del propio usuario autenticado                     | Error 400 "No puedes invitarte a ti mismo"                            | Error Controlado |
| PR2_31 | No se puede invitar a alguien ya miembro                     | Email de miembro del grupo                               | Error 400 "Este usuario ya es miembro del grupo"                      | Error Controlado |
| PR2_32 | No se puede duplicar una solicitud pendiente                 | Email con solicitud pendiente ya existente               | Error 400 "Ya existe una solicitud pendiente"                         | Error Controlado |
| PR2_33 | Solo el invitado puede aceptar la solicitud                  | Otro usuario intenta aceptar la solicitud ajena          | Error 403 Forbidden                                                   | Error Controlado |
| PR2_34 | SemaphoreSlim evita duplicados en peticiones simultáneas     | Dos peticiones simultáneas al endpoint de notificaciones | Solo se genera una notificación, la segunda encuentra la ya existente | Correcto         |
| PR2_35 | Clave primaria compuesta de Usuarios_Groups evita duplicados | INSERT de (usuarioId, groupId) ya existente              | Error de clave primaria duplicada, no se inserta                      | Error Controlado |
| PR2_36 | UNIQUE en email de Usuarios evita duplicados                 | INSERT de email ya existente                             | Error de restricción UNIQUE, no se inserta                            | Error Controlado |
| PR2_37 | FK Grupos→Usuarios es ON DELETE CASCADE                      | DELETE usuario creador del grupo                         | El grupo se elimina automáticamente                                   | Correcto         |
| PR2_38 | ErrorHandlingMiddleware captura excepción genérica           | Error inesperado en la API                               | Respuesta 500 con body { "error": "Error interno del servidor" }      | Error Controlado |

## Pruebas

|        |                                                   |                                                                |                                          |                  |
|--------|---------------------------------------------------|----------------------------------------------------------------|------------------------------------------|------------------|
| PR2_39 | Cambiar contraseña con password actual incorrecto | PUT /api/usuarios/cambiar-password con password actual erróneo | Error 401 "La contraseña no es correcta" | Error Controlado |
| PR2_40 | Conexión a Aiven requiere SSL                     | Conexión sin SSL Mode=Required                                 | La conexión es rechazada por el servidor | Error Controlado |
| PR2_41 | ObtenerPorId devuelve 403 si es de otro usuario   | GET /api/objetivos/{id} de otro usuario                        | Error 403 Forbidden                      | Error Controlado |

# Autenticación y Seguridad

La autenticación está basada en JWT (JSON Web Tokens). Al hacer login o registro, el servidor genera un token firmado con una clave secreta configurada en las variables de entorno. Este token incluye el identificador del usuario como claim y tiene una validez de 72 horas.

La aplicación móvil guarda el token en las preferencias del dispositivo y lo adjunta en la cabecera Authorization: Bearer <token> de cada petición a la API.

Todos los controladores excepto AuthController tienen el atributo [Authorize], de modo que cualquier petición sin token válido recibe automáticamente una respuesta 401

Las contraseñas nunca se almacenan en texto plano. Se usa BCrypt con salt aleatorio, lo que garantiza que incluso si dos usuarios tienen la misma contraseña el hash almacenado es diferente

El acceso a los datos está aislado por usuario. Cada consulta se filtra siempre por el usuarioid extraído del token, de modo que ningún usuario puede acceder a los datos de otro aunque conozca su identificador

# Despliegue

La API se despliega en **Railway** mediante **Docker**. El Dockerfile usa una compilación en dos etapas: la primera usa la imagen del SDK de .NET 9 para compilar y publicar la aplicación, y la segunda usa solo la imagen de runtime, más ligera, para ejecutarla. Esto reduce el tamaño final de la imagen de unos 800 MB a aproximadamente 200 MB. Cada vez que se hace un push al repositorio de GitHub, Railway detecta el cambio y se redespiega automáticamente sin hacer nosotros nada.

Railway asigna un puerto dinámico a cada contenedor mediante la variable de entorno PORT, que la API lee al arrancar. Una vez desplegada, Railway genera un dominio público que es la URL que usa la aplicación móvil para conectar con el servidor.

Toda la configuración sensible (cadena de conexión a la base de datos y clave secreta del JWT) se gestiona como variables de entorno en Railway, sin que ningún valor aparezca en el repositorio de GitHub.

La base de datos MySQL 8 está alojada en **Aiven**, un servicio gestionado en la nube. La conexión entre la API y Aiven usa SSL obligatorio, garantizando que los datos viajan cifrados

# Conclusión

FinanzasApp es un proyecto que cubre el ciclo completo de desarrollo de una aplicación real: desde el análisis de los requisitos y el diseño de la base de datos hasta la implementación del backend, la interfaz multiplataforma y el despliegue en producción. No se trata de un prototipo ni de una aplicación de demostración, sino de un producto funcional que puede usarse desde un dispositivo Android o desde un ordenador con Windows. La arquitectura en capas con cuatro proyectos bien separados ha demostrado ser una buena decisión desde el principio. Al tener el dominio aislado en Core sin dependencias externas, cualquier cambio en la infraestructura (como cambiar el ORM o el proveedor de base de datos) no afectaría a la lógica de negocio. El uso del patrón repositorio ha permitido que los controladores de la API trabajen siempre contra interfaces y no contra implementaciones concretas, lo que hace el código mucho más predecible y fácil de mantener.

El patrón MVVM aplicado en la interfaz MAUI ha sido fundamental para mantener la lógica separada de la vista. Gracias a `CommunityToolkit.Mvvm`, la generación automática de código mediante atributos como `[ObservableProperty]` y `[RelayCommand]` ha reducido significativamente el código repetitivo que normalmente requiere este patrón, permitiendo centrarse en la funcionalidad real de cada pantalla.

El despliegue en producción con Railway y Aiven ha añadido una dimensión real al proyecto que va más allá del desarrollo local. Configurar el Dockerfile con compilación en dos etapas, gestionar las variables de entorno de forma segura para que ningún secreto aparezca en el repositorio, y resolver el problema del puerto dinámico de Railway son aspectos que acercan este proyecto a lo que sería un entorno profesional real.

Como posibles mejoras futuras se identifican varias líneas de desarrollo. La primera sería la implementación de notificaciones push nativas para que las alertas lleguen al dispositivo aunque la aplicación esté cerrada. La segunda sería ampliar las funcionalidades de los grupos, añadiendo por ejemplo objetivos compartidos entre los miembros o categorías comunes del grupo. La tercera sería la exportación de datos a PDF o CSV para que el usuario pueda llevar un registro fuera de la aplicación. La cuarta sería añadir estadísticas anuales más detalladas con comparativas entre años y proyecciones. Y la quinta sería la posibilidad de adjuntar imágenes a los movimientos como comprobantes de compra o facturas.

En definitiva, FinanzasApp es un proyecto que demuestra la capacidad de desarrollar una solución completa y funcional aplicando las tecnologías y los patrones de diseño estudiados durante el ciclo, integrándose de forma coherente en un producto que tiene utilidad real para cualquier persona que quiera llevar un mejor control de sus finanzas personales.

## **Webgrafía**

- ★ **Base de Datos Aiven MySQL**
- ★ **Plataforma de Despliegue: Railway**
- ★ **API: ASP.NET Core**
- ★ **Entity Framawork Core**
- ★ **Documentación JWT**
- ★ **Documentación JWT 2**
- ★ **BCrypt.NET**
- ★ **Microsoft Clean Architecture**
- ★ **Microsoft Repository Pattern in ASP.NET Core**
- ★ **CommunityToolkit.MVVM**
- ★ **CommunityToolkit.maui**
- ★ **Microcharts**
- ★ **SkiaSharp**
- ★ **Swagger OpenAPI**